

CSCI 3656 Homework 2

Alex Gennaro

1. $4x^2 - 3x - 3$

$a=4$ $b=-3$ $c=-3$

$$\frac{-(-3) \pm \sqrt{(-3)^2 - 4(4)(-3)}}{2(4)} = \frac{3 \pm \sqrt{9+48}}{8} = \frac{3 \pm \sqrt{57}}{8}$$

$$= \begin{cases} -.568729 \\ 1.38729 \end{cases}$$

$$3.4x^2 - 3x - 3 = 0$$

$$3x = 4x^2 - 3$$

$$x = \frac{4}{3}x^2 - 1$$

Using $g(x) = \frac{4}{3}x^2 - 1$ resulted in my method oscillating around the desired point and never finding it so I had to find a different $g(x)$

$$4x^2 - 3x - 3 = 0$$

$$4x^2 = 3x + 3$$

$$x^2 = \frac{3x+3}{4}$$

$$x = \pm \sqrt{\frac{3x+3}{4}}$$

$$g(x) = \pm \sqrt{\frac{3x+3}{4}}$$

in the
bisection
method

So, I chose the interval $[-1.0, -0.5]$ to find the first solution because $f(-1.0) = 4 + 3 - 3 = 4 > 0$ and $f(-0.5) = 1 + 1.5 - 3 = -0.5 < 0$ so there must be one of the two solutions in that interval (we know there's two solutions because $f(x)$ is a quadratic with a positive a and a negative c). I chose $[1.0, 1.5]$ to find the second solution because $f(1.0) = 4 - 3 - 3 = -2 < 0$ and $f(1.5) = 9 - 4.5 - 3 = 1.5 > 0$ thus there must be...

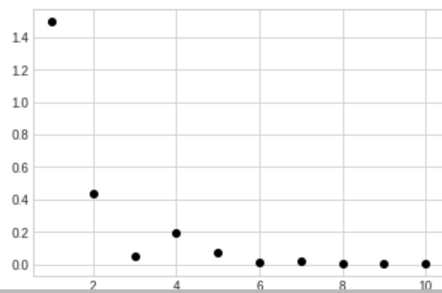
S(continued). another solution in that interval.
For my guesses for x in the fixed point method, I used the midpoints of each of those intervals, $x = -0.75$ and $x = 1.25$, because that will ensure that they are closest to the solutions in their respective intervals. Thus when the fixed point method is executed the solution we get will be the one from the desired interval.

```

import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
import math
num=0
myList = [1,2,3,4,5,6,7,8,9,10]
errorList = []
def bisect(a,b):
    c=(a+b)/2
    solution = 4*(c ** 2)-3*c-3
    if len(errorList)<10:
        errorList.append(abs(solution))
    if solution < 0.0001 and solution > -0.0001:
        return c
    elif (4*(a ** 2)-3*a-3)*solution < 0:
        return bisect(a,c)
    elif (4*(b ** 2)-3*b-3)*solution < 0:
        return bisect(c,b)
    else:
        return np.sqrt(-1)
solution1 = bisect(-1,-0.5)
solution2 = bisect(1,1.5)
print("Solution 1: %f Solution 2: %f"%(solution1,solution2))
print("Error Plot for First 10 Iterations")
plt.plot(myList, errorList, 'o', color='black');

```

Solution 1: -0.568726 Solution 2: 1.318726
Error Plot for First 10 Iterations



```

plt.style.use('seaborn-whitegrid')
myList = [1,2,3,4,5,6,7,8,9,10]
errorList = []
def g(x):
    if x > 0:
        return np.sqrt(3/4*(x+1))
    else:
        return -np.sqrt(3/4*(x+1))
def fixedpoint(x0):
    error = 9999999
    x = None
    i = 0
    while(error > 0.0001 and i < 999999):
        x = g(x0)
        error = abs(x0-x)
        if len(errorList)<10:
            errorList.append(error)
        x0 = x
        i += 1
    return x
solution1 = fixedpoint(-0.75)
solution2 = fixedpoint(1.25)
print("Solution 1: %f Solution 2: %f"%(solution1,solution2))
print("Error Plot for First 10 Iterations")
plt.plot(myList, errorList, 'o', color='black');

```

Solution 1: -0.568698 Solution 2: 1.318693
Error Plot for First 10 Iterations

